

Memilih Kuantisasi GGUF Berdasarkan RAM

https://gemini.google.com/app/d6a12252ace6c760

User prompt: mana yang abda sarankan

Response: Berdasarkan gambar file-74a6c57736a2f75c4425c9f2f7b5304a.png, pilihan yang sedang dicentang atau direkomendasikan secara umum oleh sistem (biasanya ditandai dengan ikon jempol/rekomen) adalah Q4_K_M.

Namun, pilihan terbaik untuk Anda sebenarnya sangat bergantung pada spesifikasi RAM komputer yang Anda gunakan. Berikut adalah panduan singkat untuk memilih kuantisasi GGUF yang paling sesuai:

Panduan Memilih Berdasarkan RAM Anda

Pilihan Kuantisasi	Ukuran File	Kebutuhan RAM Minimum (Sistem + Model)	Rekomendasi Penggunaan
Q3_K_L	~4.43 GB	RAM 8 GB	Pilih ini jika RAM komputer Anda pas-pasan (hanya 8 GB) agar sistem tidak <i>lag</i> atau <i>crash</i> .
Q4_K_M (Saran Utama)	~5.03 GB	RAM 8 GB - 16 GB	Sangat Direkomendasikan. Ini adalah <i>sweet spot</i> antara ukuran file, kecepatan respons (kecepatan <i>generate</i> teks), dan akurasi model yang tetap terjaga dengan baik.
Q6_K	~6.73 GB	RAM 16 GB	Pilih ini jika Anda punya RAM 16 GB dan ingin akurasi yang lebih mendekati model aslinya tanpa mengorbankan terlalu banyak performa.
Q8_0	~8.71 GB	RAM 16 GB - 32 GB	Pilih ini jika Anda mengutamakan akurasi tertinggi dan memiliki RAM atau VRAM (kartu grafis) yang besar serta longgar.

Kesimpulan

- Jika Anda ragu atau ingin performa yang seimbang antara kecepatan dan kecerdasan, **pilihlah Q4_K_M** (seperti yang sudah tercentang di gambar).
- Jika laptop/PC Anda berspesifikasi tinggi (RAM 16 GB ke atas), Anda bisa menaikkannya ke **Q6_K** untuk hasil jawaban yang sedikit lebih berkualitas.

Mana yang paling sesuai dengan spesifikasi perangkat yang Anda gunakan saat ini?

User prompt: apakah local Ai ini, yang ada di lm studio, bisa direct coding?

Response: Secara singkat: **Ya, model AI lokal di LM Studio ini bisa sangat diandalkan untuk *direct coding* (menulis, memperbaiki, dan menjelaskan kode pemrograman).**

Namun, ada satu detail penting yang perlu Anda perhatikan dari model spesifik yang ada di gambar file-74a6c57736a2f75c4425c9f2f7b5304a.png. Model yang sedang Anda buka adalah DeepSeek-R1-Distill-Qwen-8B.

Berikut adalah penjelasan bagaimana model ini bekerja untuk *coding* dan cara menggunakannya secara maksimal:

1. Kemampuan Coding Model Ini

Model berbasis **DeepSeek-R1** (meskipun versi *distilled* atau diperkecil ke 8 Billion parameter seperti di gambar) memiliki keunggulan utama pada **Reasoning (Penalaran)**.

- Kelebihan:** Sangat kuat dalam memikirkan logika *coding* yang rumit, mencari *bug*, merancang arsitektur aplikasi, atau menulis fungsi spesifik (seperti React komponen, API di Node.js/Python, dll.).
- Proses Berpikir:** Karena ini model *Reasoning*, sebelum dia memberikan baris kode, dia akan "berpikir" terlebih dahulu secara mendalam (*Chain-of-Thought*). Ini membuat kode yang dihasilkan biasanya lebih minim *error* dibanding model biasa.

2. Cara Menggunakannya untuk *Direct Coding*

Ada dua cara utama jika Anda ingin *coding* langsung menggunakan LM Studio:

- Cara 1: Lewat Chat Interface (Copy-Paste)** Anda tinggal masuk ke tab Chat di LM Studio, lalu ketik perintah seperti biasa:

"Buatlah middleware otentikasi menggunakan Next.js dan Supabase." Model akan menghasilkan kode lengkap yang bisa langsung Anda klik tombol **Copy** untuk ditempel ke VS Code atau text editor Anda.
- Cara 2: Menggunakan LM Studio sebagai Local API (Integrasi ke VS Code)** Ini adalah cara paling mendekati arti "*direct coding*" seperti GitHub Copilot atau Cursor.

1. Di LM Studio, buka tab **Local Server** (ikon panah/server di sebelah kiri).
2. Klik **Start Server**. LM Studio akan berjalan di `localhost:1234`.
3. Buka VS Code dan instal ekstensi *coding assistant* gratis seperti **Continue.dev** atau **Llama Coder**.
4. Atur ekstensi tersebut agar menyambung ke local server LM Studio Anda.
5. **Hasilnya:** Anda bisa melakukan *auto-complete* kode, *refactoring*, atau *chatting* dengan AI langsung di dalam ruang kerja VS Code Anda tanpa kuota internet.

Tips Tambahan

Karena model di gambar menggunakan basis **Qwen-8B**, performanya sudah sangat bagus untuk laptop/PC rumahan. Namun, jika nanti Anda merasa butuh model yang *jauh lebih peka* terhadap sintaksis pemrograman yang spesifik atau pustaka-pustaka model baru, Anda juga bisa mencari model bertipe "**Coder**" di kolom pencarian LM Studio (misalnya Qwen2.5-Coder-7B atau DeepSeek-Coder).

Apakah Anda berencana menggunakannya langsung lewat aplikasi LM Studio, atau ingin diintegrasikan langsung ke dalam VS Code?

User prompt: apakah agak overprepare kalau punya antigravity, devin ide, cursor, amazon q, dan juga codex, skearang coba juga lm studio

Response: Jawabannya singkat: **Iya, ini sangat overprepare dan berpotensi membuat alur kerja (workflow) Anda menjadi tumpang tindih.**

Menggunakan terlalu banyak AI *coding assistant* sekaligus justru bisa memicu *cognitive overload*—Anda akan bingung AI mana yang harus didengarkan, belum lagi konflik *shortcut* keyboard di VS Code atau editor Anda, serta pemborosan sumber daya perangkat (RAM/CPU) dan biaya langganan (jika berbayar).

Mari kita bedah dan rapikan ekosistem AI *coding* Anda agar lebih efisien:

Analisis Tools yang Anda Miliki

1. Cursor & Devin (Kategori: Autonomous Agent / Editor Utama)

- **Cursor:** Ini adalah standar emas untuk AI-native IDE saat ini. Fitur *Composer* (multi-file editing) dan *Chat*-nya sangat intuitif karena paham seluruh *codebase* Anda.
- **Devin:** Ini adalah *Autonomous AI Software Engineer*. Devin bukan sekadar asisten ketik, melainkan agen yang bisa diberikan tugas besar (misal: "tolong integrasikan API X dan perbaiki *bug* di halaman dashboard"), lalu dia akan membuka terminal, menulis kode, dan melakukan *testing* sendiri.
- **Vonis:** Cukup gunakan **Cursor** untuk *coding* sehari-hari, dan gunakan **Devin** hanya jika Anda ingin mendelegasikan tugas/fitur besar yang malas Anda kerjakan sendiri.

2. Amazon Q & CodeX (Kategori: Inline Autocomplete)

- Keduanya berfokus pada *inline completion* (menebak baris kode selanjutnya saat Anda mengetik).
- **Vonis:** Menyalakan keduanya bersamaan di VS Code/Cursor akan membuat mereka berebut menampilkan saran teks abu-abu di layar Anda. **Pilih salah satu saja**, atau matikan keduanya jika Anda sudah merasa cukup dengan fitur *Tab-autocomplete* bawaan Cursor.

3. LM Studio (Kategori: Local & Privacy-Focused)

- Berbeda dengan tools di atas yang berbasis cloud, LM Studio berjalan 100% lokal di komputer Anda menggunakan model seperti DeepSeek-R1 yang Anda buka di `file-74a6c57736a2f75c4425c9f2f7b5304a.png`.
- **Vonis:** Ini **sangat bagus sebagai opsi cadangan (backup)** atau eksperimen. Gunakan LM Studio saat Anda sedang tidak ada koneksi internet (offline), sedang mengerjakan kode yang sangat rahasia (privacy), atau ingin memanfaatkan *Reasoning* mendalam dari DeepSeek tanpa terkena *rate limit* cloud.

Rekomendasi Setup yang Lebih Ramping (Lean Setup)

Daripada mengaktifkan semuanya sekaligus, Anda bisa membaginya berdasarkan **skenario kerja** seperti ini:

- **Setup Utama (Harian):** Gunakan **Cursor** (sebagai editor utama) dikombinasikan dengan salah satu *autocomplete* cloud pilihan Anda (misal **Amazon Q** atau fitur bawaan Cursor).
- **Setup Pendelegasian (Tugas Besar):** Buka **Devin** saat Anda ingin bersantai dan membiarkan AI menyelesaikan satu modul fitur yang repetitif dari awal sampai akhir.
- **Setup Lokal / Offline:** Nyalakan **LM Studio** (hubungkan ke Cursor/VS Code via Local Server) saat kuota internet habis, sedang berada di pesawat, atau ingin *deep-dive* logika rumit menggunakan DeepSeek-R1 secara gratis tanpa batas.

Dengan membatasi tools yang aktif secara bersamaan, fokus Anda tidak akan terpecah dan Anda bisa memaksimalkan potensi dari masing-masing AI tersebut.

Dari semua tools yang sudah Anda coba di atas, fitur atau AI mana yang sejauh ini paling terasa membantu produktivitas Anda?

User prompt: *ngga sih, saya cuman pakai untuk jika kehabisan free tier*

Response: Ah, mengerti! Jadi strateginya adalah menjadikan LM Studio sebagai **jaring pengaman (safety net)** saat kuota gratisan dari *cloud tools* Anda (seperti Cursor atau Devin) habis.

Jika tujuannya seperti itu, langkah Anda justru **sangat cerdas**, bukan *overprepare*. Memiliki alternatif lokal yang 100% gratis dan tanpa batasan limit adalah taktik terbaik untuk menjaga produktivitas coding Anda tetap jalan terus tanpa gangguan tagihan bulanan.

Agar transisinya mulus saat *free tier* cloud Anda habis, berikut adalah cara mengoptimalkan LM Studio yang ada di gambar `file-74a6c57736a2f75c4425c9f2f7b5304a.png` sebagai *back-up plan*:

1. Jadikan LM Studio sebagai "Mesin Pengganti" di Editor Utama

Saat jatah *free tier* di Cursor habis, Anda tidak perlu pindah aplikasi. Anda cukup mengubah sumber AI di dalam Cursor ke server lokal Anda:

1. Di LM Studio, masuk ke tab **Local Server** lalu klik **Start Server**.
2. Buka pengaturan Cursor (atau ekstensi seperti *Continue.dev* di VS Code).
3. Masukkan URL endpoint lokal (`http://localhost:1234/v1`) dan masukkan nama modelnya: `deepseek-r1-0528-qwen3-8b`.
4. Anda bisa lanjut *coding* di editor yang sama, namun kali ini ditenagai oleh RAM dan kartu grafis Anda sendiri.

2. Keunggulan Model Pilihan Anda (DeepSeek-R1-Distill-Qwen-8B) sebagai Back-up

Model yang sedang Anda buka di gambar sangat pas untuk skenario ini karena:

- **Gratis Selamanya:** Tidak ada istilah *rate limit* atau "kuota bulanan habis". Selama laptop Anda menyala, AI ini siap melayani.
- **Kemampuan Reasoning:** Untuk tugas-tugas mencari *bug* atau memikirkan logika algoritma yang rumit, model *Reasoning* ini bahkan bisa menandingi beberapa model cloud berbayar.

Kesimpulan

Strategi Anda sudah sangat tepat. Biarkan *tools* premium cloud bekerja keras selama kuotanya masih ada, dan begitu muncul notifikasi "*Free tier limit reached*", Anda tinggal menyalakan mesin lokal di LM Studio ini dan lanjut bekerja seperti biasa.

Apakah server lokal di LM Studio-nya sudah sempat dicoba dihubungkan ke editor kode Anda?

User prompt: *hmm apa gunanya lm studio ini jika saya install lagi deepseek yang sama di ollama*

Response: Jika Anda menginstal model **DeepSeek-R1-Distill-Qwen-8B** yang sama di **Ollama**, secara teknis "otak" atau kecerdasan AI yang Anda gunakan akan **100% sama**. Keduanya sama-sama menjalankan model tersebut secara lokal menggunakan sumber daya hardware (RAM/CPU/GPU) komputer Anda.

Meskipun modelnya sama, **LM Studio** dan **Ollama** memiliki perbedaan mendasar pada cara kerja dan fitur bawaannya. Memiliki keduanya tidak selalu sia-sia, karena mereka melayani kebutuhan yang berbeda:

Perbedaan Utama: LM Studio vs Ollama

Fitur	LM Studio 	Ollama  / 
Antarmuka (UI)	Memiliki Aplikasi GUI (Grafis) yang sangat lengkap, ramah pengguna, mirip ChatGPT bawaan pabrik.	Berjalan di latar belakang (Background Service) atau lewat Terminal/Command Prompt (CLI) .
Pencarian Model	Sangat mudah. Ada <i>Hugging Face browser</i> bawaan (seperti yang Anda lihat di gambar <code>file-74a6c57736a2f75c4425c9f2f7b5304a.png</code>), tinggal cari, pilih kuantisasi, lalu klik <i>download</i> .	Harus mengetik perintah di terminal (misal: <code>ollama run deepseek-r1:8b</code>). Pilihan kuantisasinya lebih terbatas pada apa yang disediakan di library resmi Ollama.
Kontrol Parameter	Sangat Detail. Anda bisa mengatur <i>System Prompt</i> , <i>Temperature</i> , <i>Context Length</i> , hingga konfigurasi GPU <i>offloading</i> lewat tombol-tombol visual.	Pengaturan parameter harus dilakukan manual lewat file konfigurasi (<code>ModelFile</code>) yang kodenya ditulis sendiri.
Konsumsi Resource	Sedikit lebih berat karena harus menjalankan visual aplikasi (GUI) bersamaan dengan modelnya.	Sangat ringan dan efisien karena hanya berjalan sebagai <i>service</i> di latar belakang tanpa tampilan grafis.

Kapan Anda Harus Menggunakan Ollama?

Jika tujuan utama Anda adalah **direct coding / integrasi ke VS Code/Cursor**, **Ollama sering kali lebih disukai** karena:

1. **Ringan:** Dia tidak memakan RAM ekstra untuk menampilkan jendela aplikasi visual seperti LM Studio. RAM Anda bisa dialokasikan penuh untuk performa model AI dan *code editor*.
2. **Efisiensi Developer:** Ekstensi *coding* seperti *Continue.dev* atau *Llama Coder* biasanya memiliki integrasi satu-klik yang sangat mulus dengan Ollama.

Kapan LM Studio Tetap Berguna?

Anda tidak perlu menghapus LM Studio karena ia tetap punya fungsi penting:

- **Tempat Eksperimen & Playground:** Jika Anda ingin mengetes model baru, membandingkan berbagai versi kuantisasi (seperti memilih antara Q4_K_M atau Q6_K di gambar), atau sekadar ingin *chatting* santai dengan tampilan yang nyaman tanpa perlu membuka VS Code.
- **Evaluasi Cepat:** Jauh lebih mudah mencari dan mencoba model-model alternatif di luar DeepSeek lewat kolom pencarian LM Studio yang langsung terhubung ke Hugging Face.

Kesimpulan & Saran Setup

Jika Anda sudah telanjur mengunduh model yang sama di Ollama, Anda bisa menjadikannya sebagai **mesin utama di latar belakang** untuk menyuplai AI ke Cursor/VS Code Anda saat kuota cloud habis.

Sementara itu, biarkan LM Studio tetap terinstal sebagai alat *testing* atau *playground* visual Anda jika suatu saat Anda ingin berdiskusi panjang atau bereksperimen dengan parameter model secara lebih mendalam.

Apakah Anda sudah mencoba menjalankan perintah `ollama run` untuk DeepSeek ini di terminal Anda?

User prompt: ngabisin memoty doang meski memo 1TB

Response: Haha, betul banget! Meskipun memori penyimpanan (SSD/Harddisk) Anda lega sampai 1 TB, menduplikasi model yang sama di LM Studio dan Ollama itu memang terasa **mubazir kapasitas**.

Satu model berukuran 8B dengan kuantisasi Q4_K_M seperti di gambar `file-74a6c57736a2f75c4425c9f2f7b5304a.png` itu memakan ruang sekitar **5 GB**. Kalau diunduh di dua tempat berbeda, Anda membuang sekitar 10 GB hanya untuk satu jenis AI yang kemampuannya kembar identik.

Supaya SSD 1 TB Anda tidak habis untuk file duplikat yang tidak perlu, ada trik cerdas yang bisa Anda lakukan: **Gunakan satu file model saja untuk kedua aplikasi tersebut.**

Cara Menghemat Penyimpanan (Sharing Model)

Anda tidak perlu mengunduh ulang model lewat Ollama jika sudah punya filenya di LM Studio (atau sebaliknya). Anda bisa memanfaatkan fitur **Symlink (Shortcut Folder)** atau langsung mendaftarkannya via file konfigurasi.

Opsi 1: Pindahkan Semua ke Ollama, Hapus di LM Studio

Jika tujuan utama Anda adalah *coding* lewat VS Code/Cursor, **Ollama jauh lebih efisien**.

1. Cukup gunakan model yang ada di Ollama.
2. Hapus file model yang ada di LM Studio (klik ikon tempat sampah di sebelah model pada gambar Anda).
3. Jika suatu saat Anda ingin memakai UI milik LM Studio, Anda tidak perlu *download* lagi. Masuk ke pengaturan LM Studio, lalu arahkan folder *Source Models* ke folder tempat Ollama menyimpan modelnya.

Opsi 2: Gunakan Fitur Modelfile di Ollama

Jika Anda telanjur suka mencari dan mengunduh model lewat *Hugging Face browser* milik LM Studio karena fiturnya yang mudah seperti di gambar, Anda bisa menyuruh Ollama untuk membaca file bawaan LM Studio tersebut:

1. Cari lokasi file `.gguf` yang diunduh oleh LM Studio (biasanya di folder `~/ .cache/lm-studio/models/...`).
2. Buat sebuah file teks kosong di komputer Anda, beri nama `Modelfile` (tanpa ekstensi `.txt`).
3. Tulis kode ini di dalamnya:

```
FROM /jalur/ke/folder/lm-studio/nama-model-anda.gguf
```

4. Buka terminal/CMD di folder tersebut, lalu jalankan perintah:

```
```bash
ollama create nama-model-bebas -f Modelfile
```

5. **Hasilnya:** Ollama sekarang bisa menjalankan model tersebut tanpa menduplikasi atau menyalin file 5 GB itu lagi. Dia hanya membuat "jalan pintas" ke file milik LM Studio.

Dengan cara ini, kapasitas SSD 1 TB Anda tetap aman dan bisa dialokasikan untuk menyimpan aset proyek *coding* Anda yang lain. Jadi, rencana Anda sekarang mau fokus pakai aplikasi yang mana untuk cadangan?

User prompt: lah, jadi ollama bisa ngalah ke lm studio?

Response: Bisa bilang begitu! Tepatnya, **Ollama bisa "mengalah" dalam hal penyimpanan dengan cara menumpang/membaca file model yang sudah diunduh oleh LM Studio**. Jadi, Ollama tidak perlu menduplikasi file sebesar 5 GB itu lagi di foldernya sendiri.

Di dunia pemrograman, trik ini memanfaatkan kemampuan Ollama untuk membuat model kustom menggunakan perintah FROM yang diarahkan langsung ke lokasi file .gguf mentah di komputer Anda.

Karena Anda sudah mengunduh model DeepSeek-R1-Distill-Qwen-8B lewat LM Studio seperti yang terlihat di gambar file-74a6c57736a2f75c4425c9f2f7b5304a.png, Anda tinggal melakukan langkah konkret ini agar Ollama bisa langsung memakainya tanpa memakan memori tambahan:

## Langkah Praktis Menghubungkan Ollama ke File LM Studio

### 1. Salin Jalur File (.gguf) dari LM Studio

1. Buka LM Studio, pergi ke tab **My Models** (ikon folder/arsip di sebelah kiri).
2. Cari model DeepSeek-R1 yang ada di gambar.
3. Biasanya ada opsi untuk **Show on disk** atau **Copy file path**. Salin alamat lengkap lokasi file .gguf tersebut. (Contoh lokasinya biasanya seperti: C:\Users\NamaAnda\.cache\lm-studio\models\lmstudio-community\DeepSeek-R1-Distill-Qwen-8B-GGUF\deepseek-r1-0528-qwen3-8b-Q4\_K\_M.gguf).

### 2. Buat Modelfile untuk Ollama

1. Buka aplikasi Notepad atau VS Code, lalu buat file baru.
2. Tulis perintah berikut di dalamnya (ganti bagian jalur file dengan lokasi yang Anda salin tadi):

```
FROM "C:\lokasi\file\dari\lm-studio\deepseek-r1-0528-qwen3-8b-Q4_K_M.gguf"
```

3. Simpan file tersebut dengan nama **Modelfile** (pastikan hapus ekstensi .txt saat menyimpan, pilih **All Files** di

#### 3. Daftarkan ke Ollama via Terminal

1. Buka Terminal atau Command Prompt (CMD) di folder tempat Anda menyimpan Modelfile tadi.
2. Jalankan perintah ini:
 

```
```bash
ollama create deepseek-lokal -f Modelfile
```

3. Ollama akan memprosesnya dalam hitungan detik (karena dia tidak menyalin file, hanya membuat *pointer* atau jalan pintas).

Hasil Akhir

Sekarang, jika Anda mengetik perintah `ollama run deepseek-lokal`, AI tersebut akan berjalan dengan lancar lewat Ollama. Anda pun bisa langsung menghubungkannya ke Cursor atau VS Code sebagai *backup free tier* Anda.

Dengan setup ini:

- **LM Studio** bertindak sebagai tempat manajemen dan *download* model yang praktis lewat tampilan visualnya.
- **Ollama** bertindak sebagai mesin latar belakang yang hemat RAM untuk *coding* harian Anda.
- **Memori 1 TB Anda** tetap efisien karena file raksasa 5 GB tersebut hanya ada satu di sistem.

Mau langsung coba dipraktikkan untuk menghubungkannya ke editor kode Anda?

User prompt: oke kalau gitu gimana cara hapus dulu yang di ollama lalu pakai yang di lm studio?

Response: Untuk membersihkan duplikasi dan membuat Ollama "mengalah" membaca file dari LM Studio, jalankan langkah-langkah berikut secara berurutan:

Langkah 1: Hapus Model Duplikat di Ollama

Pertama, kita bersihkan dulu model yang memakan memori di folder bawaan Ollama.

1. Buka Terminal (macOS/Linux) atau Command Prompt/PowerShell (Windows).
2. Ketik perintah berikut untuk melihat nama model yang terinstal di Ollama saat ini:

```
ollama list
```

3. Cari nama model DeepSeek yang ingin Anda hapus (misalnya: `deepseek-r1:8b`).
4. Hapus model tersebut dengan perintah `ollama rm`:

```
```bash
ollama rm deepseek-r1:8b
```

(Ganti `deepseek-r1:8b` sesuai dengan nama yang muncul di daftar `ollama list` Anda).

Dengan begini, ruang penyimpanan SSD Anda langsung lega kembali karena file duplikatnya sudah terhapus.

## Langkah 2: Cari Lokasi File .gguf di LM Studio

Sekarang kita cari tahu di mana LM Studio menyimpan file `Q4_K_M` berukuran **5.03 GB** yang ada di gambar `file-74a6c57736a2f75c4425c9f2f7b5304a.png` Anda.

1. Di dalam aplikasi LM Studio, klik ikon **My Models** (ikon folder/arsip) di menu sebelah kiri.
2. Anda akan melihat daftar model yang sudah diunduh. Cari `deepseek-r1-0528-qwen3-8b`.
3. Klik tombol **Show on disk** atau ikon folder di sebelahnya untuk membuka File Explorer/Finder langsung ke lokasi file tersebut.
4. Salin alamat lengkap (*file path*) dari file `.gguf` tersebut.
  - **Contoh di Windows:** `C:\Users\NamaUser\.cache\lm-studio\models\lmstudio-community\DeepSeek-R1-Distill-Qwen-8B-GGUF\deepseek-r1-0528-qwen3-8b-Q4_K_M.gguf`
  - **Contoh di Mac/Linux:** `/Users/NamaUser/.cache/lm-studio/models/lmstudio-community/DeepSeek-R1-Distill-Qwen-8B-GGUF/deepseek-r1-0528-qwen3-8b-Q4_K_M.gguf`

## Langkah 3: Buat Jalan Pintas Menggunakan Modelfile

Langkah terakhir adalah memberi tahu Ollama untuk membaca file milik LM Studio tadi.

1. Buka aplikasi **Notepad** (Windows) atau **TextEdit** (Mac).
2. Tulis perintah satu baris ini di dalamnya (masukkan lokasi file yang Anda salin di Langkah 2 ke dalam tanda kutip):

```
FROM "C:\Lokasi\File\Anda\deepseek-r1-0528-qwen3-8b-Q4_K_M.gguf"
```

3. Simpan file tersebut dengan nama `Modelfile` (ingat: **jangan** gunakan ekstensi `.txt`. Saat menyimpan di Notepad

---

### Langkah 4: Daftarkan ke Ollama

1. Kembali ke Terminal atau Command Prompt Anda.
2. Masuk ke folder tempat Anda menyimpan `Modelfile` tadi menggunakan perintah `cd` (contoh: `cd Downloads`).
3. Jalankan perintah ini untuk mendaftarkannya:
 

```
```bash
ollama create ds-lmstudio -f Modelfile
```

4. Selesai! Prosesnya akan berjalan sangat instan (hanya hitungan detik) karena Ollama tidak menyalin file apa pun.

Cara Mengujinya

Sekarang Anda bisa langsung mengetik perintah berikut di terminal:

```
ollama run ds-lmstudio
```

Ollama akan langsung berjalan menggunakan file tunggal sebesar 5 GB milik LM Studio tanpa membuat SSD 1 TB Anda sesak lagi. Anda pun tinggal menghubungkan model `ds-lmstudio` ini ke Cursor atau VS Code sebagai cadangan gratisan Anda.

Apakah langkah pembuatan `Modelfile`-nya berjalan lancar atau ada kendala error di terminal?

User prompt: Kalau saya pakai local host model di ollama -> continue berarti saya ga perlu pakai lagi cursor dan Devin ai yang pelit?

Response: Secara teknis: **Betul sekali!**

Jika Anda menggabungkan **Ollama (dengan model DeepSeek lokal)** + ekstensi **Continue** di VS Code, Anda telah membangun ekosistem *coding assistant* mandiri yang **100% gratis, tanpa kuota, tanpa batas token, dan menjaga privasi kode Anda sepenuhnya**. Anda tidak perlu lagi bergantung pada kebijakan *free tier* dari platform cloud yang membatasi penggunaan harian Anda.

Namun, untuk memastikan ekspektasi Anda tetap terjaga dengan baik saat beralih penuh, mari kita bandingkan secara jujur apa yang Anda dapatkan dan apa yang sedikit berkurang dari hilangnya Cursor atau Devin:

Perbandingan Jujur: Setup Lokal vs Platform Cloud

1. Melawan Cursor (AI-Native IDE)

- **Kelebihan Setup Lokal Anda:** Anda kembali menggunakan VS Code standar yang familier dengan performa ringan. Tidak ada lagi kecemasan "sisa kuota saya tinggal berapa hari ini?".
- **Fitur yang Hilang:** Cursor memiliki fitur bernama **Composer** (bisa mengedit banyak file sekaligus di latar belakang secara otomatis). Ekstensi *Continue* di VS Code juga bisa membaca seluruh folder proyek Anda (*context awareness*), tetapi eksekusi modifikasi kodenya biasanya harus Anda terima dan *apply* per file secara manual.

2. Melawan Devin AI (Autonomous Software Engineer)

- **Kelebihan Setup Lokal Anda:** Devin sangat membatasi penggunaan karena biaya komputasi *agent* mereka di cloud sangat mahal. Dengan Ollama, Anda bisa melakukan *chatting* dan meminta perbaikan kode berulang-ulang tanpa memikirkan arloji waktu atau token.
- **Fitur yang Hilang:** Devin adalah sebuah **Agent**. Dia bisa membuka terminal sendiri, menjalankan *build compiling*, membaca error log, lalu memperbaikinya sendiri sampai aplikasi Anda jalan. Sementara kombinasi Ollama + Continue bertindak sebagai **Asisten**. AI memberikan saran kode terbaik lewat panel chat atau *inline autocomplete*, lalu Anda yang mengeksekusi dan menjalankan kodenya di terminal.

Tips Memaksimalkan Setup "Ollama + Continue" Anda

Agar pengalaman *coding* lokal Anda terasa sekuat menggunakan *tools* premium cloud tersebut, pastikan melakukan optimasi ini di pengaturan ekstensi **Continue**:

1. **Gunakan Fitur @ (Context Providers):** Saat sedang *chatting* dengan DeepSeek di panel Continue, ketik @Files atau @Folder untuk memasukkan file atau seluruh direktori proyek Anda ke dalam memori AI. Ini meniru kemampuan pemahaman *codebase* milik Cursor.
2. **Manfaatkan Fitur Cmd+I atau Ctrl+I:** Fitur ini memungkinkan Anda memblokir baris kode di VS Code, lalu meminta DeepSeek mengubah atau merefaktorisasi kode tersebut langsung di tempat (Inline Edit).
3. **Manfaatkan Reasoning DeepSeek-R1:** Karena model di file-74a6c57736a2f75c4425c9f2f7b5304a.png adalah tipe *Reasoning*, saat Anda meminta logika yang rumit, biarkan panel Continue menampilkan proses berpikirnya (*thinking process*). Hasil kodenya biasanya akan jauh lebih matang dibanding model biasa.

Kesimpulan

Keputusan untuk beralih ke **Ollama + Continue** adalah langkah merdeka terbaik bagi seorang developer yang lelah dengan batasan *free tier*. Anda mendapatkan kendali penuh atas alat kerja Anda sendiri.

Apakah konfigurasi ekstensi Continue di VS Code Anda sudah berhasil terhubung dengan model ds-lmstudio yang kita buat tadi?

User prompt: I mean saya bisa pakai *antigravity* yang lebih generous saja, lalu continue di vscode secara terpisah

Response: Ah, paham maksud Anda! Jadi strateginya adalah membagi tugas: menggunakan **Antigravity** yang kuota gratisannya jauh lebih melimpah atau murah hati, lalu jika sedang ingin menghemat kuota tersebut atau saat sedang *offline*, Anda tinggal berpindah ke **Continue + Ollama** di VS Code secara terpisah.

Strategi kombinasi ini justru adalah **langkah taktis terbaik**. Anda tidak membuang aset memori lokal Anda, tetapi juga tidak membatasi diri hanya pada kemampuan hardware lokal saat proyek Anda sedang butuh bantuan model *cloud* raksasa.

Dengan skenario kerja terpisah seperti itu, alur kerja Anda akan menjadi sangat rapi:

Alur Kerja Kombinasi Ideal Anda

1. Skenario Utama (Fitur Kompleks & Eksplorasi): Pakai Antigravity

- Gunakan kelonggaran kuota dari Antigravity untuk tugas-tugas berat di awal, seperti mendesain arsitektur aplikasi baru, melakukan *brainstorming* logika yang sangat rumit, atau meminta penjelasan dokumentasi *framework* terbaru yang mungkin belum masuk ke database model lokal Anda.
- Manfaatkan kelebihanannya selama jatahnya masih longgar.

2. Skenario Pendukung (Eksekusi Rutin & Debugging): Pakai Continue + Ollama

- Saat Anda hanya butuh bantuan mengetik fungsi-fungsi sederhana (seperti membuat fungsi *formatting* tanggal, komponen UI kecil, atau menulis *unit test*), gunakan **Continue** dengan model ds-lmstudio di VS Code.
- Gunakan juga setup lokal ini saat sedang melakukan *debugging* eror kecil berulang-ulang, agar Anda tidak membuang kuota Antigravity hanya untuk membetulkan typo atau salah *import*.

Keuntungan Setup Terpisah Ini

- **Bebas dari Rasa Pelit Cloud Platform:** Anda benar-benar bisa mencoret Cursor atau Devin dari daftar jika sistem kuota mereka membuat proses *coding* Anda menjadi tidak nyaman dan penuh tekanan.
- **Manajemen Resource yang Efisien:** Anda hanya menyalakan Ollama saat membutuhkan Continue. Saat menggunakan Antigravity, RAM laptop Anda bisa fokus 100% untuk menjalankan aplikasi dan *build tool* proyek Anda tanpa beban model lokal.

- **Keamanan Data Mutlak:** Jika suatu saat Anda menyentuh proyek yang berisi data sensitif, kunci API rahasia, atau kode privat, Anda tinggal mematikan koneksi internet dan beralih penuh ke Continue + Ollama demi privasi total.

Kombinasi ini membuat Anda memiliki *setup* yang mandiri dan tidak bisa "Disetir" oleh kebijakan platform mana pun. Apakah Anda sudah selesai mengatur *Modelfile* di Ollama dan siap menguji performa Continue untuk mendampingi Antigravity?

User prompt: Kalau saya setiap n8n, apakah dia hanya akan kerja ketika laptop nyala? Kalau di docker apakah saya harus start docker setiap saat?

Response: Secara singkat: **Ya, jika n8n diinstal di laptop/PC lokal (baik langsung maupun lewat Docker), dia hanya akan bekerja ketika laptop Anda menyala dan aplikasi Docker-nya aktif.**

Jika laptop Anda mati, tertidur (*Sleep/Hibernate*), atau aplikasi Docker Desktop-nya ditutup, maka seluruh alur kerja otomatisasi (*workflow*) yang sudah Anda buat di n8n akan ikut **berhenti total** dan tidak bisa menerima pemicu (*trigger*) apa pun.

Mari kita bahas bagaimana cara kerja Docker di laptop dan solusinya agar n8n Anda bisa berjalan otomatis:

1. Apakah Harus Start Docker Setiap Saat?

Secara bawaan (*default*), Anda memang harus membuka Docker Desktop setiap kali laptop baru dinyalakan agar kontainer n8n Anda berjalan. Namun, Anda bisa membuat proses ini **100% otomatis** sehingga Anda tidak perlu repot klik *Start* manual setiap saat.

Ada dua pengaturan yang harus Anda aktifkan:

- **Pengaturan di Docker Desktop (Windows/Mac):** Masuk ke *Settings* (ikon gerigi) -> *General* -> Centang pilihan **"Start Docker Desktop when you log in"**. Sekarang, begitu laptop menyala dan Anda masuk ke Windows/Mac, Docker akan otomatis berjalan di latar belakang.
- **Pengaturan di Kontainer n8n (Restart Policy):** Saat Anda menjalankan perintah Docker untuk n8n di terminal, pastikan Anda menambahkan perintah `--restart always`. Contoh perintah lengkapnya:

```
docker run -d --name n8n -p 5678:5678 --restart always -v ~/.n8n:/home/node/.n8n n8nio/n8n
```

Fungsinya: Dengan `--restart always`, setiap kali Docker Desktop menyala (atau setelah laptop Anda *reboot*), Docker akan otomatis langsung menghidupkan kontainer n8n Anda di latar belakang tanpa perlu Anda klik tombol *Play* manual.

2. Solusi Jika n8n Harus Kerja 24 Jam Tanpa Mati

Jika Anda membuat otomatisasi yang sifatnya kritis (misalnya: harus membalas pesan bot Telegram instan, mengecek *database* crypto setiap jam, atau *scraping* data terjadwal), mengandalkan laptop pribadi tentu kurang ideal karena laptop tidak mungkin dinyalakan terus-menerus.

Solusi terbaiknya adalah memindahkan Docker n8n Anda dari laptop ke **Cloud Server (VPS)**.

Metode	n8n di Laptop (Docker Lokal)	n8n di Cloud Server (VPS seperti AWS, DigitalOcean, dll.)
Biaya	100% Gratis.	Berbayar bulanan (mulai dari \$4 - \$5/bulan).
Ketersediaan	Hanya jalan saat laptop hidup & terkoneksi internet.	Aktif 24/7 tanpa henti , meskipun laptop dan HP Anda mati total.
Cocok Untuk	Eksperimen, membangun <i>workflow</i> , otomatisasi internal saat jam kerja.	Otomatisasi bisnis, bot <i>real-time</i> , integrasi webhook pihak ketiga.

Kesimpulan

Untuk saat ini, jika Anda masih dalam tahap coba-coba dan membangun alur kerja, pakai saja **Docker lokal di laptop** dengan mengaktifkan fitur *Start on login* dan *Restart Policy Always* di atas. Jadi begitu Anda membuka laptop untuk *coding*, n8n sudah langsung siap digunakan di `localhost:5678`.

Apakah otomatisasi yang sedang Anda rancang di n8n ini membutuhkan respon *real-time* 24 jam, atau hanya dipakai saat Anda sedang aktif bekerja di laptop saja?

User prompt: Run docker itu boros baterai ga?

Response: Secara singkat: **Iya, menjalankan Docker di laptop (terutama di Windows atau Mac) tergolong cukup boros baterai.**

Jika Anda sering bekerja menggunakan daya baterai (tanpa dicolok ke listrik), membiarkan Docker Desktop menyala terus-menerus di latar belakang akan membuat persentase baterai laptop Anda turun jauh lebih cepat daripada biasanya.

Berikut adalah alasan teknis mengapa Docker boros baterai dan cara menyiasatinya:

Mengapa Docker Boros Baterai?

1. **Menjalankan Virtual Machine (VM) Tersembunyi** Docker dibuat asli untuk sistem operasi Linux. Ketika Anda menjalankannya di Windows (lewat WSL2) atau di Mac, Docker sebenarnya meluncurkan sebuah sistem Linux virtual mini di latar belakang. Menjalankan dua sistem operasi sekaligus (OS utama Anda + Linux VM Docker) membutuhkan daya CPU dan RAM yang konstan.
2. **Proses Latar Belakang (*Background Services*)** Meskipun Anda tidak sedang membuka *workflow* n8n di browser, kontainer Docker dan *engine*-nya tetap melakukan pengecekan berkala (seperti memeriksa status kontainer, alokasi memori, atau sinkronisasi file). Aktivitas konstan ini mencegah CPU laptop Anda masuk ke mode hemat daya (*deep sleep state*).
3. **Aplikasi n8n yang Selalu Siap Sedia** Aplikasi seperti n8n memiliki proses *internal timer* untuk mengecek apakah ada jadwal otomatisasi yang harus berjalan. Proses pengecekan konstan ini membutuhkan daya komputasi secara berkala.

Tips Menghemat Baterai Tanpa Menghapus Docker

Jika Anda ingin baterai laptop tetap awet saat dibawa bepergian, Anda bisa menerapkan strategi berikut:

- **Matikan Docker Saat Tidak Dipakai** Jangan biarkan Docker Desktop menyala jika Anda sedang tidak membutuhkan n8n atau sedang fokus menulis kode biasa. Cukup klik kanan ikon Docker di *system tray* (pojok bawah kanan) lalu pilih **Quit Docker Desktop**.
- **Jangan Aktifkan "Start Docker on Login" Jika Sering Mobile** Jika Anda sering bekerja di luar ruangan tanpa *charger*, matikan fitur otomatis menyala saat laptop hidup. Nyalakan Docker secara manual hanya saat Anda benar-benar ingin menguji atau menggunakan n8n.
- **Gunakan Perintah Terminal untuk Menghentikan Kontainer** Jika Anda ingin Docker Desktop tetap menyala tetapi ingin mengistirahatkan n8n agar tidak memakan daya, Anda bisa menghentikan kontainernya sementara lewat terminal dengan perintah:

```
docker stop n8n
```

Dan ketika ingin meneruskan kerja, tinggal ketik:

```
```bash
```

```
docker start n8n
```

## Kesimpulan

Docker sangat bagus untuk produktivitas Anda, tetapi dia adalah komponen yang "lapar" daya. Gunakan Docker secara taktis: **nyalakan saat mode charging (dicolok listrik) untuk membangun alur kerja n8n, dan matikan aplikasinya saat Anda harus menghemat baterai di perjalanan.**

*User prompt: Kalau Dnegan n8n, kita bisa connect proyek yang terhubung ke supabase, kita connect ke Excel/sheets, mana yang lebih mudah? N8n atau make.com karena saya kurang paham make.com kemairn*

**Response:** Karena Anda kemarin merasa kurang paham dengan Make.com, maka jawabannya sangat jelas: **n8n akan terasa jauh lebih mudah dan masuk akal untuk Anda**, terutama karena proyek Anda melibatkan **Supabase**.

Mari kita bedah perbandingannya secara objektif mengapa n8n lebih unggul dan lebih mudah untuk kebutuhan Anda saat ini:

## Mengapa n8n Lebih Mudah untuk Kasus Anda?

### 1. Integrasi Supabase di n8n Jauh Lebih Ramah Developer

- **Di n8n:** n8n memiliki *node* bawaan (*native node*) khusus untuk Supabase. Anda cukup memasukkan URL proyek dan API Key Supabase Anda. Setelah itu, Anda bisa melakukan operasi seperti *Insert*, *Update*, *Delete*, atau *Get* data dari tabel Supabase Anda cukup dengan klik-klik menu visual.
- **Di Make.com:** Make.com sering kali tidak memiliki *node* bawaan yang matang untuk *database* modern seperti Supabase. Anda biasanya terpaksa harus menggunakan *node* HTTP Generic untuk menembak REST API Supabase secara manual, yang artinya Anda harus menyusun *header*, *authentication*, dan *body* JSON sendiri. Ini sangat membingungkan kalau Anda belum terbiasa.

### 2. Logika Berpikir n8n Lebih Linear dan Mirip Coding

- Tampilan n8n bergerak dari **kiri ke kanan** dengan struktur *node-based* yang sangat mirip dengan logika diagram alir (*flowchart*) atau fungsi *coding*.
- Bagi seseorang yang terbiasa dengan konsep pemrograman, membaca aliran data di n8n jauh lebih intuitif. Anda bisa melihat dengan jelas data apa yang keluar dari Supabase dan bagaimana data itu masuk ke Google Sheets/Excel.
- Sementara itu, Make.com menggunakan visual lingkaran-lingkaran yang memutar dan sistem *routing* yang kadang terasa asing jika Anda memiliki pola pikir berbasis logika *coding*.

### 3. Manipulasi Data Menggunakan JavaScript (Mulus di n8n)

- Saat menghubungkan Supabase ke Excel/Sheets, format datanya tidak selalu langsung cocok. Di n8n, jika Anda butuh mengubah atau menyaring data, Anda cukup menambahkan **Code Node** dan menulis beberapa baris JavaScript sederhana untuk merapikannya.
- Di Make.com, Anda dipaksa menggunakan fungsi bawaan (*built-in formulas*) mereka yang memiliki sintaksis tersendiri (seperti `{{get(map(...))}}`) yang sering kali membingungkan dan sulit di-*debug* jika terjadi eror.

## Alur yang Akan Anda Buat di n8n (Sangat Sederhana)

Untuk menghubungkan Supabase ke Google Sheets atau Excel di n8n, alurnya hanya membutuhkan 2 sampai 3 *node* saja:

[Supabase Node] —> [Code Node (Opsional)] —> [Google Sheets Node]  
 (Mengambil data dari tabel Anda) (Merapikan format data) (Memasukkan data ke baris baru)

1. **Supabase Node (Trigger/Action):** Mengambil baris data baru dari tabel Supabase Anda.
2. **Google Sheets / Excel Node:** Memilih file *spreadsheet* Anda, lalu memetakan kolom (misal: Kolom ID di Supabase masuk ke Kolom A di Sheets, Nama ke Kolom B, dst).

## Catatan Penting Mengenai Excel vs Google Sheets

Jika Anda ingin setup yang **paling mudah tanpa pusing**, gunakan **Google Sheets** di n8n. Koneksinya sangat stabil dan autentikasinya mudah (tinggal hubungkan akun Google).

Jika Anda menggunakan Excel (Microsoft 365), proses pengaturan izin di Microsoft Azure Portal terkadang sedikit lebih ribet di awal, meskipun setelah tersambung fungsinya akan sama saja.

## Kesimpulan

Lanjutkan saja eksplorasi Anda di **n8n**. Karena n8n berjalan lokal di Docker Anda, Anda bisa mencoba membuat *workflow* Supabase ke Sheets ini puluhan kali sehari secara gratis tanpa perlu takut kehabisan kuota *operations* seperti di Make.com.

Apakah Anda sudah menyiapkan tabel di Supabase dan *file* di Google Sheets yang ingin dihubungkan sekarang?

User prompt: jawab ringkas, kok saya ga bisa matikan fitur reasoning pada deepseek qwen 8B saya di lm studio?

Response: Jawabannya ringkas: **Fitur Reasoning pada DeepSeek-R1 (termasuk versi Qwen 8B di gambar file-74a6c57736a2f75c4425c9f2f7b5304a.png) tidak bisa dimatikan karena itu sudah "tertanam" keras di dalam otak modelnya (hardcoded).**

Berikut 2 alasan utamanya:

1. **Bukan Fitur Software:** *Reasoning* (<think>...</think>) bukan setelan sakelar di aplikasi LM Studio yang bisa di-klik *on/off*. Itu adalah hasil dari proses *training* modelnya. Model tersebut dididik untuk **wajib** mengeluarkan proses berpikir sebelum menjawab.
2. **Jika Dipaksa Berhenti:** Anda bisa saja menyembunyikan tampilannya di UI LM Studio atau menyuruhnya lewat *System Prompt* ("jangan berpikir"), tetapi hal itu justru akan merusak kualitas jawabannya dan membuatnya sering error.

**Solusinya:** Jika Anda ingin model 8B yang cepat dan langsung menjawab tanpa proses *Reasoning*, Anda harus mengunduh model biasa yang bukan versi R1, seperti *Qwen2.5-7B-Instruct* atau *Llama-3-8B-Instruct* lewat LM Studio.

User prompt: apakah localhosted ai yang di lm studio tidak bebas edit file di folder> kalau yang di ollama gimana?

Response: Secara bawaan (*default*), **baik LM Studio maupun Ollama TIDAK BISA mengedit, mengubah, atau membuat file baru di dalam folder komputer Anda secara langsung.**

Kedua alat ini sifatnya terisolasi (hanya menerima teks masukan, lalu mengeluarkan teks jawaban di layar).

Namun, ada perbedaan besar ketika Anda menghubungkannya ke editor kode seperti VS Code:

## 1. Jika Lewat Aplikasi LM Studio / Ollama Langsung

- **LM Studio:** Anda hanya bisa melakukan *chat* di dalam aplikasinya. Anda harus *copy-paste* kodenya secara manual ke dalam folder proyek Anda.
- **Ollama:** Sama saja. Jika dijalankan lewat terminal (`ollama run`), dia hanya mengeluarkan teks di terminal tersebut.

## 2. Bagaimana Cara Membuatnya Bisa "Bebas Edit File"?

Agar AI lokal ini bisa langsung mengedit file di folder proyek Anda secara otomatis (seperti fitur Cursor), Anda harus **menghubungkannya ke ekstensi pihak ketiga di VS Code.**

Di sinilah **Ollama jauh lebih diunggulkan** dibanding LM Studio:

- **Ollama + Ekstensi (Sangat Direkomendasikan):** Jika Anda menghubungkan Ollama ke ekstensi seperti **Continue.dev** atau **Aider** di VS Code, ekstensi inilah yang akan memberikan "izin akses" ke folder Anda.
  - Melalui ekstensi ini, model DeepSeek dari Ollama bisa langsung membaca file, melakukan *refactor*, dan **menulis/mengubah kode langsung di dalam file folder Anda** saat Anda menyetujui kodenya (*Apply Changes*).
- **LM Studio + Ekstensi:** Secara teknis bisa dilakukan dengan menyalakan *Local Server* di LM Studio. Namun, karena LM Studio memakan RAM lebih besar untuk tampilan aplikasinya, performa komputer saat AI sedang membaca dan mengedit banyak file di folder akan terasa lebih berat dan lambat dibanding menggunakan Ollama yang berjalan ringan di latar belakang.

## Kesimpulan

Jika Anda ingin AI lokal yang bisa **langsung mengedit file di folder**, gunakan **Ollama** yang disambungkan ke ekstensi **Continue.dev** di VS Code. Kombinasi itu adalah cara terbaik untuk meniru kemampuan edit file otomatis milik Cursor secara 100% lokal dan gratis.

*User prompt: apa maksudnya token ini, bukannya free limitation?*

Response: Berdasarkan gambar dyn-d921eda2f164e1b4274c64c3ef924861.png, Anda tidak perlu khawatir! Ini **bukan batasan kuota gratis (free tier/limitation)** dari sebuah platform cloud berbayar. LM Studio 100% gratis dan berjalan lokal di komputer Anda.

Maksud dari angka token di gambar tersebut adalah **Batasan Memori Konteks (Context Window Limitation)** hardware Anda saat ini.

Berikut penjelasan mudahnya:

### 1. Apa itu "Context Window"?

- *Context Window* adalah kapasitas "ingatan jangka pendek" AI dalam satu sesi obrolan.
- Pada gambar, batasnya diatur di **2048 token** (kira-kira setara dengan 1.500 kata).
- Saat ini, obrolan Anda baru memakai **372 token** (18.2%).

### 2. Apa yang Terjadi Kalau Menyentuh 2048 Token (100%)?

AI tidak akan mengunci Anda atau meminta bayaran. Namun, AI akan mulai "**pikun**". Dia akan melupakan instruksi atau kode di bagian paling atas obrolan agar bisa mengingat pesan-pesan yang paling baru dari Anda.

### 3. Mengapa Batasnya Kecil Sekali (Hanya 2048)?

Secara bawaan (*default*), LM Studio sering kali mengatur *Context Length* di angka 2048 agar beban RAM/VRAM laptop Anda tetap ringan.

Namun, karena Anda menggunakan model **DeepSeek R1** yang membutuhkan ruang ekstra untuk menuliskan proses berpikirnya (*Reasoning*), batas 2048 token ini **pasti akan cepat habis** saat dipakai *coding*.

### Cara Menaikkan Batas Ingatannya (Gratis & Mudah)

Anda bisa memperbesar ingatan AI ini agar bisa membaca kode yang lebih panjang:

1. Lihat ke panel menu sebelah kanan di LM Studio (saat berada di tab Chat).
2. Cari bagian bernama **Hardware Settings** atau **Context Length**.
3. Ubah angka 2048 menjadi lebih besar, misalnya **4096** atau **8192** (sesuaikan dengan sisa RAM/VRAM laptop Anda).

Jadi, aman ya! Itu murni batasan konfigurasi *software* lokal, bukan kuota premium yang habis.

*User prompt: kalau ra 16gb bisa sampai berapa?*

Response: Jika laptop Anda memiliki **RAM 16 GB**, Anda aman untuk menaikkan kapasitas ingatan konteksnya hingga **4096** atau **8192 (8K) token**.

Untuk model berukuran 8 Billion parameter (seperti DeepSeek-R1-Qwen-8B yang Anda gunakan), berikut adalah gambaran konsumsi RAM berdasarkan panjang konteksnya:

- **4096 Token (Sangat Aman & Direkomendasikan):** Memerlukan sekitar 6 GB - 7 GB RAM untuk modelnya saja. Sisa RAM laptop Anda masih sangat lega untuk membuka VS Code, beberapa tab browser Chrome, dan sistem operasi tanpa membuat laptop macet.
- **8192 Token (Maksimal untuk Kenyamanan Multitasking):** Konsumsi RAM akan naik ke sekitar 8 GB - 9 GB. Ini masih sangat bisa dijalankan di RAM 16 GB, namun jika Anda membuka proyek *coding* yang sangat besar atau menjalankan Docker n8n secara bersamaan, sisa RAM laptop Anda akan mulai menipis.

### Cara Mengubahnya di LM Studio:

1. Di panel menu sebelah kanan (tab Chat), cari bagian **Context Length** atau **Context Window**.
2. Ketik manual atau pilih angka **4096** (jika ingin laptop tetap super ringan untuk multitasking) atau **8192** (jika butuh AI mengingat kode proyek yang panjang).
3. Cari juga pengaturan bernama **GPU Offload** (jika laptop Anda memiliki kartu grafis/VRAM dedikasi, naikan nilainya agar sebagian beban RAM dipindahkan ke kartu grafis sehingga performanya jauh lebih cepat).

Untuk kebutuhan *backup coding* harian Anda, angka **4096** sampai **8192** sudah lebih dari cukup untuk menampung beberapa file kode sekaligus tanpa membuat AI cepat lupa.

